Storefront Masterclass: 10-Module Curriculum

Build: a web + mobile storefront where sellers open a store, list items and get paid. Guests can buy without an account.

Stack: Next.js 15 (static export) · Supabase (DB, Auth, Storage, Edge Functions) · Paystack · WhatsApp Cloud API · Resend · Capacitor · Vercel · GitHub Actions

Owners: N = Noah: Supabase, auth, payments, Resend/WhatsApp, domain and launch (Modules 1, 2, 4, 5, 9). **C** = Co-founder: storefront UI, Capacitor, native features, CI/CD, assessment (Modules 3, 6, 7, 8, 10).

#	Module	What gets built	Owner	End state
1	Setup, Supabase & Keeping Secrets Safe	Repo, Next.js app with `output: "export"`, Supabase project, tables (`stores`, `products`, `orders`, `order_items`), Row Level Security, `.env.local` and Vercel env vars, buy domain and verify it with Resend, set up Claude safely with SQL migrations and a dev project	N	Empty app talking to the DB, secured, domain and email ready, no secrets in the repo
2	Seller Accounts & Store Creation	Supabase Auth, store profile, add/edit products with image upload	N	A seller can sign up and list an item
3	Storefront, Cart & Guest Checkout	Public store page, product listing, cart state, checkout with no login, account prompt only at pay	C	A guest can fill a cart and reach checkout
4	Payments & Orders	Paystack checkout, Edge Function webhook to verify payment, order saved as paid, seller sees orders	N	A real test-mode payment creates a paid order
5	Notifications & Verification	WhatsApp OTP at checkout, order alerts to seller and buyer, Resend receipt emails	N	Sale triggers WhatsApp + email
6	Wrap the Web App with Capacitor (Android)	Add Capacitor, build the Android app, run it on a device	C	App runs on a real Android phone
7	Native Features & iOS	Push notifications (FCM), splash and icons, deep links, iOS build and TestFlight	C	Sale sends a push to the seller's phone
8	CI/CD with GitHub Actions	Lint, test and build on every push, Android build artifact, Vercel deploy, secrets handling	C	Push to `main` deploys the web app and builds the APK
9	Vercel Production Launch	Attach the domain to Vercel, production env vars, switch Paystack to live, Supabase prod settings	N	Live on the real domain
10	Onboarding Assessment & Go-Live	Students sign up on the live app and complete the assessment; Terms and Privacy published	C	Students onboarded on the MVP

Key handling rules (taught in Module 1)

- **Assume the app bundle is public.** Anything with a `NEXT_PUBLIC_` prefix is bundled into the website and the APK. Encryption can't hide it, because the app must hold the decryption key.
- **Only three values ship in the app:** the Supabase URL, the Supabase anon key and the API URL. The anon key is designed to be public; Row Level Security is what protects the data.
- **Plain reads and writes** (products, cart, store profile) go straight to Supabase with the anon key and Row Level Security.
- **Sensitive actions** (create order, verify payment, send OTP, send email) go through Supabase Edge Functions, which check the user's login token. Edge Functions must allow the Capacitor origins (`https://localhost`, `capacitor://localhost`) in CORS.
- **Secrets live only in Edge Function secrets** (`supabase secrets set`): the service-role key, Paystack secret key, Resend key and WhatsApp token. They never go in the app, the repo or Vercel.
- **Locally:** public values in `.env.local` (gitignored), with a blank `.env.example` committed. **On Vercel:** the same public values as environment variables.
- **Claude and the database:** Claude writes SQL migrations that are applied with the Supabase CLI, against a separate dev project. It never gets the service-role key, and a deny rule stops it reading `.env*` files.
- **Safety nets:** GitHub push protection and secret scanning, and what to do if a key leaks (rotate it in the dashboard immediately).

Decisions and risks to settle before recording

- **Web framework:** Next.js with `output: "export"`. No server-rendered pages: use client-side routing for stores and products, and put webhooks (Paystack) in Supabase Edge Functions.
- **WhatsApp API:** Meta's Cloud API needs a business account, verification and approved message templates. This can take days or weeks. Start now, and decide on a fallback (SMS or email OTP) if approval stalls.
- **Resend free tier:** your notes say about 300 emails/month. I believe Resend's free plan is higher (about 3,000/month, 100/day). Please check their current pricing page.
- **"No Android SDK needed":** you still need Android build tools to produce an APK. GitHub Actions can do the build for students (Module 8), but Module 6 needs Android Studio or the SDK locally.
- **iOS needs a Mac and a paid Apple Developer account** (about $99/yr). Android Play Store is a one-time $25. Say this up front so students aren't surprised.
- **Paystack:** test mode is fine for Modules 4–8. Going live needs business verification, so start it early. Decide how sellers get paid (split payments or subaccounts) before Module 4.
- **Seller dashboard:** kept minimal (orders list only) to stay at 10 modules. Anything bigger is a bonus module.